

1. 실행 화면 캡처

```
● (.venv) → day1_종합실습 git:(main) python collect_async.py
3개 API 동시 수집 완료: 1.11초

--- open_meteo ---
{'latitude': 37.55, 'longitude': 127.0, 'generationtime_ms': 0.048041343688964844, 'utc_offset_seconds': 32400, 'timezone': 'Asia/Seoul',
'timezone_abbreviation': 'GMT+9', 'elevation': 34.0, 'hourly_u

--- country_info ---
{'area': 100210, 'cioc': 'KOR', 'flag': '🇰🇷', 'gini': 31.4, 'name': 'Korea (Republic of)', 'flags': {'png': 'https://flagcdn.com/w320/kr.png', 'svg': 'https://flagcdn.com/kr.svg'}, 'latlng': [37, 127].

--- ip_api ---
{'status': 'success', 'country': 'United States', 'countryCode': 'US', 'region': 'VA', 'regionName': 'Virginia', 'city': 'Ashburn', 'zip': '20149', 'lat': 39.03, 'lon': -77.5, 'timezone': 'America/New

assert 통과: 3개 API 응답 정상 확인
```

```
● (.venv) → day1_종합실습 git:(main) python validate_schema.py
=== open_meteo: WeatherRecord 검증 ===
전체 72건 중 valid 72건, errors 0건
예시 (첫 3건): [{'time': '2026-07-15T00:00', 'temperature_2m': 25.1, 'precipitation_probability': 100}, {'time': '2026-07-15T01:00', 'temperature_2m': 25.1, 'precipitation_probability': 100}, {'time': '2026-07-15T02:00', 'temperature_2m': 25.2, 'precipitation_probability': 100}]

=== country_info: CountryInfo 검증 ===
검증 성공: {'name': 'Korea (Republic of)', 'capital': 'Seoul', 'region': 'Asia', 'population': 51780579, 'area': 100210.0}

=== ip_api: IpInfo 검증 ===
검증 성공: {'query': '8.8.8.8', 'country': 'United States', 'city': 'Ashburn', 'lat': 39.03, 'lon': -77.5, 'status': 'success'}

=== 데모: 일부러 잘못된 데이터로 예외 처리 확인 ===
temperature_2m='abc'(타입 오류) + precipitation_probability=150(범위 초과) -> 검증 실패 확인
2 validation errors for WeatherRecord
temperature_2m
  Input should be a valid number, unable to parse string as a number [type=float_parsing, input_value='abc', input_type=str]
  For further information visit https://errors.pydantic.dev/2.13/v/float_parsing
precipitation_probability
  Input should be less than or equal to 100 [type=less_than_equal, input_value=150, input_type=int]
  For further information visit https://errors.pydantic.dev/2.13/v/less_than_equal
```

```
● (.venv) → day1_종합실습 git:(main) python save_compare.py

=== weather (open_meteo) (72행, timeit 20회 평균) ===
쓰기: CSV 0.265ms vs Parquet 0.341ms
읽기: CSV 0.235ms vs Parquet 0.434ms
파일 크기: CSV 1,821B vs Parquet 3,408B

=== country_info (1행, timeit 20회 평균) ===
쓰기: CSV 0.161ms vs Parquet 0.325ms
읽기: CSV 0.240ms vs Parquet 0.464ms
파일 크기: CSV 85B vs Parquet 3,313B

=== ip_info (1행, timeit 20회 평균) ===
쓰기: CSV 0.158ms vs Parquet 0.331ms
읽기: CSV 0.210ms vs Parquet 0.435ms
파일 크기: CSV 84B vs Parquet 3,720B

=== cProfile: weather 저장/읽기 1회 사이클 프로파일링 (누적 시간 상위 8개) ===
3377 function calls (3344 primitive calls) in 0.002 seconds

Ordered by: cumulative time
List reduced from 522 to 8 due to restriction <8>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
1      0.000    0.000    0.002    0.002  /Users/js/skala/data-project/day1_종합실습/save_compare.py:87(run_once)
1      0.000    0.000    0.001    0.001  /opt/homebrew/lib/python3.11/site-packages/pandas/core/frame.py:3020(to_parquet)
1      0.000    0.000    0.001    0.001  /opt/homebrew/lib/python3.11/site-packages/pandas/io/parquet.py:409(to_parquet)
1      0.000    0.000    0.001    0.001  /opt/homebrew/lib/python3.11/site-packages/pandas/core/generic.py:3795(to_csv)
1      0.000    0.000    0.001    0.001  /opt/homebrew/lib/python3.11/site-packages/pandas/io/parquet.py:508(read_parquet)
1      0.000    0.000    0.001    0.001  /opt/homebrew/lib/python3.11/site-packages/pandas/io/parsers/readers.py:349(read_csv)
1      0.000    0.000    0.001    0.001  /opt/homebrew/lib/python3.11/site-packages/pandas/io/parquet.py:174(write)
1      0.000    0.000    0.001    0.001  /opt/homebrew/lib/python3.11/site-packages/pandas/io/parsers/readers.py:258(_read)

[sys.getsizeof] weather_rows (list of dict, 72건): 664 bytes
[sys.getsizeof] weather_df (pandas DataFrame): 3,044 bytes
(list는 dict 객체들의 포인터만 세고 dict 내부 값은 shallow하게 안 잡히는 반면, DataFrame은 실제 컬럼 데이터를 연속 메모리 블록에 들고 있어 계속값의 성격이 다르다.)
참고: 행 수가 적으면 (수십~수백 건) 파일 오픈/파싱 오버헤드가 지배적이라 CSV와 Parquet 차이가 크지 않거나 역전될 수 있다. Parquet의 이점(컬럼형 압축, 빠른 읽기)은 수만~수백만 행 규모에서 뚜렷해진다.
```

```
● (.venv) → day1_종합실습 git:(main) python3 -m ruff check .
All checks passed!
```

텍스트를 입력하세요.

```
● (.venv) → day1_종합실습 git:(main) pytest -q
.....
13 passed in 0.05s
```

2. Code 분석 결과에 대한 의견

가장 심각한 건 `save_compare.py`에서 크래시가 나는 두 곳이다. `country_info`나 `ip_api` 검증이 실패하면 `validate_one()` 함수가 `None`을 돌려주는데, 여기서는 그걸 체크 안 하고 바로 `model_dump()`를 호출해서 죽는다. `validate_schema.py`에서는 똑같은 상황에 `if country_valid:` 로 체크하면서 정작 `save_compare.py`에서는 빠뜨렸다. 비슷하게 `open_meteo` 요청이 실패하면 `weather_df`가 빈 `DataFrame`이 되는데, 이걸 CSV로 썼다가 다시 읽으면 `EmptyDataError`가 난다. 직접 재현해서 확인했다.

그 외에 조용히 잘못되는 부분들도 있었다. `ip-api`는 실패해도 HTTP 200을 보내는 경우가 있는데 (`status: fail`) 지금 코드는 이것 성공으로 착각해서 "응답 정상 확인" `assert`가 거짓으로 통과해버린다. `weather` 검증에서 실패한 행은 그냥 버려져서 몇 개가 왜 빠졌는지 알 수 없고, 시간대별 배열을 `zip()`으로 묶는 부분도 길이가 안 맞으면 에러 없이 조용히 잘린다. 예외 처리도 `ValidationError`만 잡고 있어서 API 응답 형태가 아예 바뀌면(과거 `RestCountries`처럼) 못 잡는다.

나머지는 자잘한 것들이다. `docstring`에 이제 없는 파일명이 그대로 남아있고, 같은 파일을 두 번 쓰는 불필요한 코드가 있고, 세 스크립트가 각자 API를 다시 호출해서 순서대로 돌리면 같은 API를 세 번씩 부른다. 검증 로직도 두 함수에서 거의 똑같이 중복돼 있다.

전체적으로 보면 정상적으로 돌아가는 경로는 신경 써서 만들었는데(`assert`로 각 단계 검증하고, `Pydantic`으로 타입 체크하고, 벤치마크할 때 `pyarrow` 콜드스타트 문제도 직접 잡아냈다), 뭔가 실패했을 때 처리하는 부분은 파일마다 들쭉날쭉했다. 같은 함수를 두 파일에서 각자 불러 쓰다 보니 한쪽엔 넣은 방어 코드가 다른 쪽엔 빠진 게 문제의 원인 같다.

고칠 부분은 이렇다.

- 1) `save_compare.py`에 `None` 체크 추가하는 게 제일 급함
- 2) `weather_df`가 비었을 때 그냥 죽지 않고 에러 메시지 내고 종료
- 3) `ip-api` 같은 "200인데 사실은 실패"인 응답도 잡히게 수정
- 4) 버려지고 있는 `validate` 에러들도 로그 남기기
- 5) API를 매번 새로 부르지 말고 한 번 받은 결과를 재사용

앞으로 더 개선하고 싶은 점은 아래와 같다.

- validate_one / validate_many를 하나로 합쳐서 중복 없애기
- collect_async.py의 except Exception을 좀 더 구체적인 예외로 좁히기
- 함수에 타입 힌트를 다 붙이고 mypy도 돌려보기
- 로그를 파일로도 남기기
- 데이터 양을 늘려서 Parquet 벤치마크를 다시 해보기

성능 측정 결과

save_compare.py로 CSV/Parquet에 저장하고 timeit 20회 평균으로 쓰기/읽기 시간을 잰다.

구분	쓰기(CSV/Parquet)	읽기(CSV/Parquet)	파일크기(CSV/Parquet)
weather(72행)	0.40ms / 0.48ms	0.34ms / 0.55ms	1,821B / 3,408B
country_info(1행)	0.17ms / 0.31ms	0.20ms / 0.41ms	85B / 3,313B
ip_info(1행)	0.15ms / 0.28ms	0.19ms / 0.40ms	84B / 3,720B

메모리 비교(sys.getsizeof): weather_rows(list, 72건) 664B, weather_df(DataFrame) 3,044B. list는 포인터만 세고 DataFrame은 실제 데이터를 들고 있어서 계측 성격이 다르다.